

Appendix 2

Debug Examples

1. Dry Run Example

Function: correct_answer()

Purpose: Ensure multiple clicks do not increase score incorrectly.

Steps:

1. Duck starts at self.duck = Duck(self.canvas, 32, 455, "../img/duck2.png")
2. Score = 0
3. Player clicks correct answer → correct_answer() runs:
 - Checks self.question_locked → False
 - Updated score = 1
 - Sets self.question_locked = True
 - Duck moves toward cracker → self.duck.move(45, self.cracker, on_collision=on_eaten)

Observation: Only one increment per question

The trace table below outlines the movement and positioning of the sprites (the duck and the cracker), also when the collision happens. It also includes the automation of the collision logic to show the cracker being hidden upon the triggering of on_eaten.

Trace Table – Duck Movement on Correct Answer

Function: correct_answer() → triggers self.duck.move(45, self.cracker, on_collision=on_eaten)

Step	Duck X	Duck Y	Cracker X	Cracker Y	Collision ?	Notes
1	32	455	91	447	No	Duck starts at specified x,y position, cracker is visible at first x,y position
2	37.8	455	91	447	No	Duck moves right, bouncing animation starts
3	43.6	455	91	447	No	Duck continues moving, velocity and gravity bouncing variables applied
4	45	455	91	447	Yes	Duck reaches cracker → collision detected, on_eaten() called
5	45	455	None	None	N/A	Cracker hidden after collision, feedback cleared, next question loaded

Using the duck movement code the columns show the position of the duck compared to the cracker and if a collision has taken place:

- Duck X / Duck Y: Current position of the duck on the canvas.
- Cracker X / Cracker Y: Current position of the cracker sprite. None after it is hidden.
- Collision?: Whether the duck collides with the cracker. Uses `check_collision()` logic